

No AI was used

1. State Matrix

Each control state is styled once. Valid and invalid feedback uses the :user-valid / user-invalid pseudo classes, so it appears only after the user interacts with a control. Untouched option fields are never shown as errors.

- Default is shown as resting control with a 1px border on a white surface.
- Focus is shown as a 3px outline with a 2px offset via :focus-visible.
- Required is shown with a label that carries a visible asterisk (aria-hidden) with a visually hidden (“required”) for screen readers.
- Valid is shown as a green border once a field has been interacted with and its values satisfies the rule.
- Invalid is shown as a red border and a pale-pink fill once a field has been interacted with and its value breaks the rule.
- Disabled is shown as a muted grey with a not allowed cursor via :disabled.
- Server-returned error is shown by aria-invalid=”true” and the .field-error block. Uses three cues, never color alone.

2. Fictional test data

- Contact email: vince@example.com
- Reference number: NW-100237
- Product area: App
- Request category: Bug
- Priority: High
- Preferred contact method: Email
- Preferred appointment date: 2026 – 9 -20
- Preferred time: Morning (8am – 12pm)
- Affected users: 5
- Issues description: App crashes when exporting a report to pdf
- Send status updates by email: Checked

?email=vince%40example.com&reference=NW-100237&product=app&category=bug&priority=high&contact_pref=email&preferred_date=2026-09-20&time_window=morning&affected_count=5&description=App+crashes+when+exporting+a+report+to+pdf&status_updates=yes

Variant Values for submission Matrix

- Empty submission: all fields blank
- Malformed email: vince.example
- Boundary-minimum: affected users = 1; date = 2026 -09 – 13, Earliest allowed
- Boundary-maximum: affected users = 50
- Out of range: Is rejected would be affected users = 0 and 51 or higher; date = 2026 -09 -12.

3. Test Record

HTML Validation: index.html

Nu Html Checker

This tool is an ongoing experiment in better HTML checking, and its behavior remains subject to change

Showing results for contents of text-input area (checked with vnu 26.9.9)

Checker Input

Show

☐ source

☐ outline

☐ image report

☐ errors & warnings only

Options...

Check by

text input

☐ check as CSS

```
<div>
  <p class="specimen-label">disabled</p>
  <button type="button" disabled>disabled example/button>
</div>
<section id="documentation" aria-labelledby="documentation-heading">
  <h2 id="documentation-heading">2. Planning and Evidence</h2>
  <p>Supporting documentation for the prototype.</p>
  <div>
    <div href="docs/evidence.pdf">evidence document (PDF)</div> - state matrix, fictional test data, test record, and all disclosure.</div>
  </div>
</section>
</body>
</html>
```

Check

Document checking completed. No errors or warnings to show.

field-inventory-template.html

Showing results for contents of text-input area (checked with vnu 26.9.9)

Checker Input

Show

☐ source

☐ outline

☐ image report

☐ errors & warnings only

Options...

Check by

text input

☐ check as CSS

```
<tr>
  <td>Send status updates</td>
  <td>Opt-in convenience, off by default.</td>
  <td>No</td>
  <td><code>input type="checkbox"</code></td>
  <td><code>status_updates</code> | <code>yes</code></td>
  <td>N/A</td>
  <td>Optional, unchecked by default.</td>
  <td>Consent / opt-in, default off to minimize messaging.</td>
</tr>
</tbody>
</table>
</body>
</html>
```

Check

Document checking completed. No errors or warnings to show.

CSS Validator:

W3C CSS Validator results for TextArea (CSS level 3 + SVG)

Congratulations! No Error Found.

This document validates as [CSS level 3 + SVG](#) !

To show your readers that you've taken the care to create an interoperable Web page, you may displ



Keyboard only completion and submission

Navigated with Tab, arrow keys for radio groups and selects. Space to toggle the checkbox, and Enter to submit.

Navigated in logical order. The disabled documentation specimens are skipped. Empty submit is blocked and focus moves to the email field.

F3 - Requester Contact

Contact email *

Format: name@example.com

vince@example.com

Reference number

Optional | Example: NW-123456

NW-100237

F3 - Request details

Product area *

Select a product area

Select a product area

App

Website

API

4. Accessible name and group inspection

- Text (email): accessible name “Contact email (required)”, described by “Format: name@example.com”.
- Radio group (priority): group name “Priority (required)” with options Low, Normal, High, Urgent.
- Checkbox: “Send status updates by email”.
- Select (product): “Product area (required)”.
- Textarea (description): “Issue description (required)”, described by its guidance hint.

☐ Urgent

Preferred contact method *

☐ Email

☐ Phone

☐ Chat

F3 - Scheduling

Preferred appointment date *

mm/dd/yyyy

Preferred time *

Please select a time window that works for you.

Select a time window

F3 - Description & preferences

ARIA tree

```
aria-describedby: preferred_date-help
aria-label: Not specified
from label (for= attribute) Label: "Preferred appointment date (required)"
title: Not specified
Description: "On or after September 13, 2026."
Role: date
Invalid user entry: true
Focusable: true
Can set value: true
```

5. Submission Matrix

- Empty: submission blocked; browser flags the first required control (email).
- Malformed email (vince.example): rejected (type mismatch).
- Boundary-minimum (affected = 1, date = 2026-09-13): accepted.
- Boundary-maximum (affected = 50): accepted; 0 and 51 rejected (range under/overflow); date 2026-09-12 rejected.
- Valid (full dataset): accepted; page reloads with the query string

6. Viewport checks

320px

Dimensions: Responsive 320 x 675 Fit to window No throttling 'Save-Data': default

Fields marked * are required.

F3 - Requester Contact

Contact email *
Format: name@example.com

vince@example.com

Reference number
Optional | Example: NW-123456

NW-100237

F3 - Request details

Product area *
Select a product area

Request category *

Dimensions: Responsive 768 x 675 Fit to window No throttling 'Save-Data': default

Contact email *
Format: name@example.com

vince@example.com

Reference number
Optional | Example: NW-123456

NW-100237

F3 - Request details

Product area *
Select a product area

768px

1280px

Dimensions: Responsive 1280 x 750 Fit to window No throttling 'Save-Data: default'

vince@example.com

Reference number
Optional | Example: NW-123456

NW-100237

F3 - Request details

Product area *
Select a product area

Request category *
Select a request category

Priority *
☐ Low
☐ Normal
☐ High

7. 200% zoom

Fields marked * are required.

F3 - Requester Contact

Contact email *
Format: name@example.com

vince@example.com

Reference number
Optional | Example: NW-123456

NW-100237

8. Focus visibility

F3 · Requester Contact

Contact email *
Format: name@example.com

Reference number
Optional | Example: NW-123456

9. Query String inspection

?email=vince%40example.com&reference=NW-100237&product=app&category=bug&priority=high&contact_pref=email&preferred_date=2026-09-20&time_window=morning&affected_count=5&description=App+crashes+when+exporting+a+report+to+pdf&status_updates=yes

Code defense

Task and scope (F1). I built a bounded service-request prototype: a customer books a support session. The scope is deliberately narrow, with one outcome which kept the field set focused and every control justifiable.

Data minimization (F2). I collected only what the task needed. There is no name, phone, or address field; the single piece of personal data is a contact email, the one unavoidable channel for a reply, and I justify it explicitly in the field inventory. Optional fields (reference number, affected users, status updates) are optional by design so requesters aren't forced to over-share, and the description field warns against pasting private information.

Semantic structure (F3). The form uses 11 purposeful controls across four <fieldset>s with real <legend>s, and eight native control types (email, text, select, radio, date, number, textarea, checkbox). Every control has a persistent <label> tied by for/id; radio groups sit in their own labelled fieldsets; submission uses an explicit <button type="submit">. Selects use a disabled, empty-valued placeholder so “required” is meaningful and the field genuinely fails an empty submit.

Validation (F4). type="email", the date min, and the number min/max/step used with no rigid pattern on the reference field, since the brief warns against unnecessary rigidity. Every requirement is stated before entry via a help paragraph linked with aria-describedby, so users know the rule before they type.

States and recovery (F5). Valid/invalid styling uses :user-valid/:user-invalid, so feedback appears only after interaction and untouched optional fields are never flagged as errors. The static server-error specimen signals the error with three cues, never color alone. A red border/fill, a text “Error” tag, and a written recovery message preserves the user's entry, and links back to the field from an error summary.

System and accessibility (F6). I merged my Module 1 design system (reset, tokens, type scale, and components) into one styles.css and adapted it, restyling the header banner and controls so no starter styling remains. Accessibility decisions include a visually-hidden “(required)” for screen readers behind the visible asterisk, a :focus-visible outline on every control, and a layout that reflows without horizontal scroll at 320px and 200% zoom.